

数据库系统原理

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第3章 关系数据库标准语言SQL



厚德 笃学 崇实 尚新

第三章 SQL语言

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 查询

3.5 数据更新

3.6 空值的处理

3.7 视图



3.5 数据更新

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据



3.5 .1 插入数据

(1) 插入元组

```
INSERT INTO <表名> [(<属性列1>[, <属性列2 >...])  
VALUES (<常量1> [, <常量2>] ... );
```

(2) 插入子查询结果

```
INSERT INTO <表名> [(<属性列1> [, <属性列2>... ]) 子查询;
```

注意： 数据更新操作中的完整性检查！

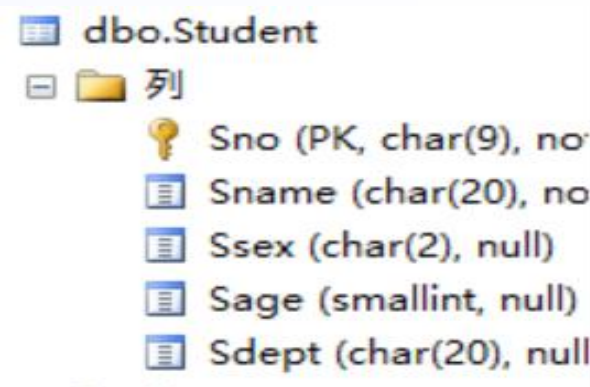


插入单个元组

[例1] 将学生元组（学号：20155128；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中。

```
INSERT INTO Student (Sno, Sname, Ssex, Sdept, Sage)
```

```
VALUES ('20155128', '陈冬', '男', 'IS', 18);
```



插入单个元组

[例1] 将学生陈冬的信息插入到Student表中。

INSERT INTO Student

VALUES ('20155128', '陈冬', '男', 'IS', 18);



列	数据类型	是否主键	是否空
Sno	char(9)	是	否
Sname	char(20)	否	否
Ssex	char(2)	否	否
Sage	smallint	否	是
Sdept	char(20)	否	否

INSERT INTO Student

VALUES ('20155128', '陈冬', '男', 18 , 'IS');



(1 行受影响)



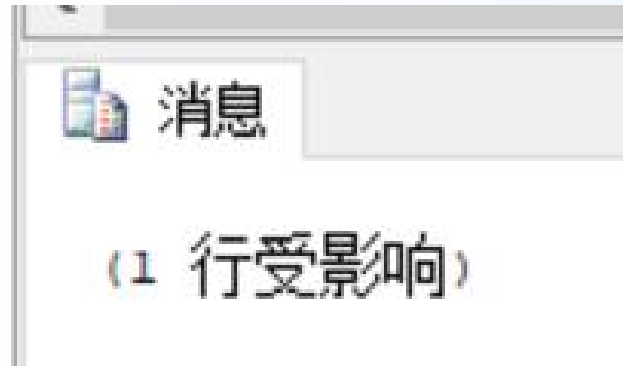
插入单个元组

[例2] 插入一条选课记录('20155128', '1')。

INSERT INTO SC(Sno, Cno) VALUES (' 20155128 ', ' 1 ');

或:

INSERT INTO SC VALUES (' 20155128 ', ' 1 ', NULL);



Sno	Cno	Grade
20155128	1	NULL
NULL	NULL	NULL

消息 2627, 级别 14, 状态 1, 第 1 行
违反了 PRIMARY KEY 约束“PK__SC__E60002530DAF0CB0”。不能在对象“dbo.SC”中插入重复键。重复键值为 (20155128 , 1)。
语句已终止。



插入单个元组（续）

■ INTO子句

- 指定要插入数据的表名及属性列
- 属性列的顺序可与表定义中的顺序不一致
- 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
- 指定部分属性列：插入的元组在其余属性列上取空值

■ VALUES子句

- 提供的值必须与INTO子句匹配
 - > 值的个数
 - > 值的类型



插入子查询结果

[例3] 对每一个系，求学生的平均年龄，并把结果存入数据库。

第一步：建表

```
CREATE TABLE Dept_age (Sdept CHAR(15), Avg_age SMALLINT);
```

第二步：插入数据

```
INSERT INTO Dept_age(Sdept, Avg_age)
SELECT Sdept, AVG(Sage) FROM Student
GROUP BY Sdept;
```

Sdept	Avg age
CS	20
MA	18
IS	19
AI	18
(NULL)	(NULL)



插入子查询结果（续）

➤ INTO子句(与插入单条元组类似)

- ★ 指定要插入数据的表名及属性列
- ★ 属性列的顺序可与表定义中的顺序不一致
- ★ 没有指定属性列：表示要插入的是一条完整的元组
- ★ 指定部分属性列：插入的元组在其余属性列上取空值

➤ 子查询

- ★ **SELECT**子句目标列必须与**INTO**子句匹配
 - 值的个数
 - 值的类型



插入子查询结果（续）

DBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则

➤ 实体完整性

➤ 参照完整性

➤ 用户定义的完整性

- ★ 对于有**NOT NULL**约束的属性列是否提供了非空值
- ★ 对于有**UNIQUE**约束的属性列是否提供了非重复值
- ★ 对于有值域约束的属性列所提供的属性值是否在值域范围内

```
INSERT INTO SC(Sno,Cno) VALUES ('20155129','1');
```

消息 547, 级别 16, 状态 0, 第 1 行
INSERT 语句与 FOREIGN KEY 约束"FK__SC__Sno__0F975522"冲突。该冲突发生于数据库"EDUC", 表"dbo.Student", column 'Sno'。
语句已终止。



3.5 数据更新

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据



3.5.2 修改数据

3.5.2 修改数据

UPDATE <表名> SET <列名>=<表达式>[, <列名>=<表达式>]...
[WHERE <条件>];

✓ **功能：** 修改指定表中满足WHERE子句条件的元组。

✓ **方式：**

- (1) 修改某一个元组的值
- (2) 修改多个元组的值
- (3) 带子查询的修改



3.5 .2 修改数据

[例4] 将学生20155121的年龄改为22岁。

```
UPDATE Student SET Sage=22 WHERE Sno=' 20155121 ';
```

[例5] 将所有学生的年龄增加1岁。

```
UPDATE Student SET Sage= Sage+1;
```



3.5 .2 修改数据

[例6] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC SET Grade=0
WHERE 'CS'=
      (SELETE Sdept FROM Student
       WHERE Student.Sno = SC.Sno);
```

■ 或

```
UPDATE SC SET Grade=0
WHERE sno IN
      (SELETE Sno FROM Student
       WHERE Sdept = 'CS' );
```



修改数据（续）

■ SET子句

指定修改方式

要修改的列

修改后取值

■ WHERE子句

指定要修改的元组

缺省表示要修改表中的所有元组

■ DBMS在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则

- ✓ 实体完整性
- ✓ 参照完整性
- ✓ 用户定义的完整性

- **NOT NULL**约束

- **UNIQUE**约束

- 值域约束



3.5.3 删除数据

3.5.3 删除数据

DELETE FROM <表名> [WHERE <条件>];

■ 功能

- 删除指定表中满足WHERE子句条件的元组。

■ WHERE子句

- 指定要删除的元组;
- 缺省表示要删除表中的所有元组，表的定义仍在字典中。

■ 三种删除方式

- 删除某一个元组的值
- 删除多个元组的值
- 带子查询的删除语句



3.5.3 删除数据

[例7] 删除学号为20155128的学生记录。

```
DELETE FROM Student WHERE Sno= ' 20155128 ';
```

[例8] 删除所有的学生选课记录。

```
DELETE FROM SC;
```

[例9] 删除计算机科学系所有学生的选课记录。

```
DELETE FROM SC  
WHERE 'CS'=  
      (SELETE Sdept FROM Student  
       WHERE Student.Sno=SC.Sno);
```

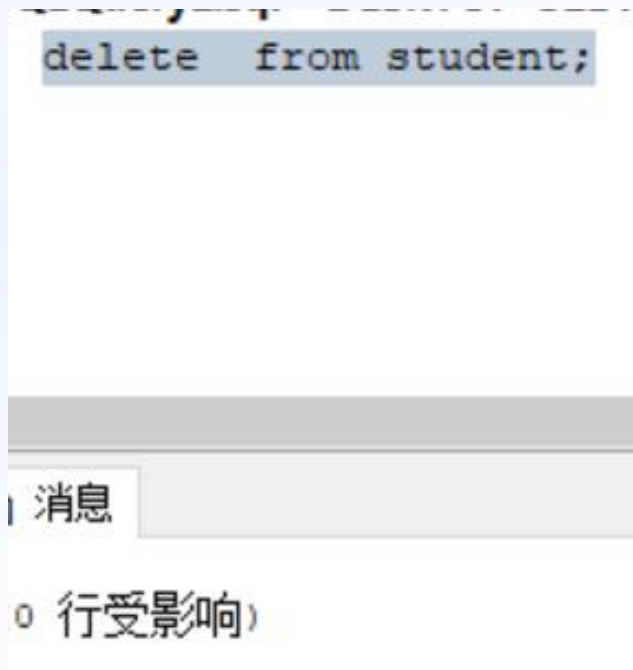


删除数据(续)

DBMS在执行删除语句时会检查所插元组是否破坏表上已定义的完整性规则

— 参照完整性

- 不允许删除
- 级联删除



更新数据与数据一致性

DBMS在执行插入、删除、更新语句时必须保证数据库一致性

- 必须有事务的概念和原子性（后面学习）
- 完整性检查和保证



第三章 SQL语言

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 查询

3.5 数据更新

3.6 空值的处理

3.7 视图



3.6 空值的处理

■ NULL的不确定性

- “不知道”、“不存在”、“无意义”
- 一般有以下几种情况：
 - ✓ 该属性应该有一个值，但目前不知道它的具体值
 - ✓ 该属性不应该有值
 - ✓ 由于某种原因不便于填写



1.空值的产生

- 空值是一个很特殊的值，含有不确定性。对关系运算带来特殊的问题，需要做特殊的处理。
- 空值的产生有其实际需求
学生在选课后，产生选课表，但是还没有成绩。这时候成绩部分就为空值，它和0不一样（不是0分）

[例10] 向SC表中插入一个元组，学生号是“201215126”，课程号是“1”，成绩为空。

[例11]将Student表中学生号为“201215200”的学生所属的系改为空值。

```
UPDATE Student SET Sdept=NULL WHERE Sno= '201215200';
```



2.空值的判断及约束条件

■ 空值的判断

- ✓ 判断一个属性的值是否为空值，用IS NULL或IS NOT NULL来表示

[例10] SELECT * FROM Student WHERE Sname IS NULL OR Ssex IS NULL OR Sage IS NULL OR Sdept IS NULL;

■ 空值的约束条件

- ✓ 属性定义（或者域定义）中有NOT NULL约束条件的不能取空值
- ✓ 加了UNIQUE限制的属性不能取空值
- ✓ 码属性不能取空值



空值的算术运算、比较运算和逻辑运算空值

- 空值与另一个值（包括另一个空值）的算术运算的结果为空值
- 空值与另一个值（包括另一个空值）的比较运算的结果为**UNKNOWN**
- 有**UNKNOWN**后，传统二值（**TRUE**，**FALSE**）逻辑就扩展成了三值逻辑

表3.8 逻辑运算符真值表

x	y	x AND y	x OR y	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

T表示TRUE，F表示FALSE，U表示UNKNOWN



空值的算术运算、比较运算和逻辑运算空值

[例10]找出选修1号课程的不及格的学生。

```
SELECT Sno
FROM SC
WHERE Grade <60 AND Cno='1';
```

	Sno
1	20155129

Sno	Cno	Grade
20155128	1	NULL
20155129	1	50
NULL	NULL	NULL

[例11]选出选修1号课程的不及格的学生以及缺考的学生。

```
SELECT Sno
FROM SC
WHERE Grade <60 AND Cno ='1'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='1';
```

	Sno
1	20155128
2	20155129

```
SELECT Sno
FROM SC
WHERE Cno='1'AND(Grade<60 OR Grade IS NULL);
```

	Sno
1	20155128
2	20155129



第3章 关系数据库标准语言SQL

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 查询

3.5 数据更新

3.6 空值的处理

3.7 视图



3.7 视图

■ 视图

- ✓ 虚表，是从一个或几个基本表（或视图）导出的表
- ✓ 只存放视图的定义，不存放视图对应的数据
- ✓ 基表中的数据发生变化，从视图中查询出的数据也随之改变
- ✓ 基于视图的操作：

查询、删除、受限更新、定义基于该视图的新视图



3.7 视图

3.7.1 定义视图

3.7.2 查询视图

3.7.3 更新视图

3.7.4 视图的作用



创建视图

■ 创建视图

CREATE VIEW

<视图名> [(<列名> [, <列名>]...)]

AS <子查询>

[WITH CHECK OPTION];

- ✓ 组成视图的属性列名：全部省略或全部指定。
- ✓ 子查询不允许含有**ORDER BY**子句和**DISTINCT**短语。
- ✓ RDBMS执行**CREATE VIEW**语句时只将视图定义存入数据字典，并不执行其中的**SELECT**语句。
- ✓ 视图查询时，其数据来源于相关基本表。



创建视图

[例1] 建立信息系学生的视图。

```
CREATE VIEW IS_Student
AS
SELECT Sno, Sname, Sage FROM Student
WHERE Sdept= 'IS';
```

[例2] 建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student
AS
SELECT Sno, Sname, Sage FROM Student
WHERE Sdept= 'IS'
WITH CHECK OPTION
```

■ WITH CHECK OPTION

透过视图进行增删改操作时，不得破坏视图定义中的谓词条件
(即子查询中的条件表达式)

- 若一个视图是从单个基本表导出，我们称这类视图为行列子视图。

IS_Student视图就是一个行列子集视图。

主



创建视图

[例3] 建立信息系选修了1号课程的学生视图。（基于多个表的视图）

```
CREATE VIEW IS_S1(Sno, Sname, Grade) AS
```

```
SELECT Student.Sno, Sname, Grade
```

```
FROM Student, SC
```

```
WHERE Sdept= 'IS' AND Student.Sno=SC.Sno AND SC.Cno= '1';
```

[例4] 建立信息系选修了1号课程且成绩在90分以上的学生视图。（基于视图的视图）

```
CREATE VIEW IS_S2 AS
```

```
SELECT Sno, Sname, Grade
```

```
FROM IS_S1
```

```
WHERE Grade>=90;
```



创建视图

[例5] 定义一个反映学生出生年份的视图。（带表达式的视图）

```
CREATE VIEW BT_S(Sno, Sname, Sbirth) AS  
  
SELECT Sno, Sname, 2023-Sage FROM Student;
```

[例6] 将学生的学号及他的平均成绩定义为一个视图。（分组视图）

```
CREATE VIEW S_G(Sno, Gavg) AS  
  
SELECT Sno, AVG(Grade) FROM SC GROUP BY Sno;
```

[例7] 将Student表中所有女生记录定义为一个视图。（不指定属性）

```
CREATE VIEW F_Student(F_Sno, name, sex, age, dept) AS  
  
SELECT * FROM Student WHERE Ssex='女' ;
```



Sno	Gavg
202215121	88.3333
202215122	85.0000
202215126	(NULL)
(NULL)	(NULL)

➤ 一类不易扩充的视图,以 **SELECT *** 方式创建的视图可扩充性差，应尽可能避免



删除视图

■ 删除视图

DROP VIEW <视图名> **[CASCADE]**;

- ✓ 该语句从数据字典中删除指定的视图定义。
- ✓ 如果该视图上还导出了其他视图，使用**CASCADE**级联删除语句，把该视图和由它导出的所有视图一起删除。
- ✓ 删除基表时，由该基表导出的所有视图定义都必须显式地使用**DROP VIEW**语句删除。

[例8] 删除视图IS_S1

DROP VIEW IS_S1;



3.7 视图

3.7.1 定义视图

3.7.2 查询视图

3.7.3 更新视图

3.7.4 视图的作用



■ 查询视图

- 用户角度：查询视图与查询基本表相同
- RDBMS实现视图查询的方法：

视图消解法：

- ✓ 进行有效性检查；
- ✓ 转换成等价的对基本表的查询；
- ✓ 执行修正后的查询。

实体化视图：

- ✓ 有效性检查；
- ✓ 执行视图定义，生成临时表；
- ✓ 查询视图转化为查询临时表；
- ✓ 查询完毕后删除实体化得视图

。



查 询 视 图

[例9] 在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT Sno, Sage FROM IS_Student WHERE Sage<20;
```

视图消解转换后的查询语句为:

```
SELECT Sno, Sage FROM Student  
WHERE Sdept= 'IS' AND Sage<20;
```

[例10] 查询选修了1号课程的信息系学生。

```
SELECT IS_Student.Sno, Sname FROM IS_Student, SC  
WHERE IS_Student.Sno =SC.Sno AND SC.Cno= '1';
```



查 询 视 图

■ 视图消解法的局限

有些情况下，视图消解法不能生成正确查询。采用视图消解法的DBMS会限制这类查询。

[例11] 在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩。

```
SELECT * FROM S_G WHERE Gavq>=90;
```

S_G视

CR

目前多数关系数据库系统对行列子集视图的查询均能进行正确转换。但对非行列子集视图的查询（如例11）就不一定能做转换了，因此这类查询应该直接对基本表进行。

✗: SELECT

WHERE

✓: SELECT Sno,

GROUP BY Sno

WHERE子句中是不能用聚集函数作为条件表达式的，因此执行此修

正后的查

```
SELECT *  
FROM (SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno) AS S_G(Sno,Gavg)  
WHERE Gavg>=90;
```



3.7 视图

3.7.1 定义视图

3.7.2 查询视图

3.7.3 更新视图

3.7.4 视图的作用



更新视图

- 用户角度：更新视图与更新基本表相同
- DBMS实现视图更新的方法
 - 视图实体化法（View Materialization）
 - 视图消解法（View Resolution）

[例12] 将信息系学生视图IS_Student中学号20155122的学生姓名改为“刘辰”。

```
UPDATE IS_Student SET Sname= '刘辰'  
WHERE Sno= ' 20155122 ';
```

转换后的语句：

```
UPDATE Student SET Sname= '刘辰'  
WHERE Sno= ' 20155122 ' AND Sdept= 'IS';
```



更新视图

[例13] 向信息系学生视图IS_Student中插入一个新的学生记录：20155129，赵新，20岁。

```
INSERT INTO IS_Student VALUES( '20155129' , '赵新' ,  
20);
```

转换为对基本表的更新：

```
INSERT INTO Student(Sno, Sname, Sage, Sdept)  
VALUES( '200215129 ' , '赵新' , 20, 'IS' );
```

[例14] 删除信息系学生视图IS_Student中学号为20155129的记录。

```
DELETE FROM IS_Student WHERE Sno= ' 20155129 ';
```

转换为对基本表的更新：

```
DELETE FROM Student WHERE Sno= ' 20155129 ' AND Sdept= ' IS' ;
```



更新视图

■ 更新视图的限制:

- ✓ 一些视图是不可更新的，这些视图的更新无法转换成对相应基本表的更新。

例如：视图S_G为不可更新视图。

```
UPDATE S_G SET Gavg=90 WHERE Sno= '20155121';
```

- ✓ 允许对行列子集视图进行更新
- ✓ 对其他类型视图的更新, 不同系统有不同限制



3.7 视图

3.7.1 定义视图

3.7.2 查询视图

3.7.3 更新视图

3.7.4 视图的作用



视图的作用

■ 视图的作用

1. 视图能够简化用户的操作；
2. 视图使用户能以多种角度看待同一数据；
3. 视图对重构数据库提供了一定程度的逻辑独立性；
 - ★ 由于对视图的更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变。
4. 视图能够对机密数据提供安全保护；
 - ✓ 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据
 - ✓ 通过WITH CHECK OPTION对关键数据定义操作时间限制
5. 适当的利用视图可以更清晰的表达查询。



视图的作用

■ 视图能够简化用户的操作

- 当视图中数据不是直接来自基本表时，定义视图能够简化用户的操作
- ✓ 基于多张表连接形成的视图
- ✓ 基于复杂嵌套查询的视图
- ✓ 含导出属性的视图

■ 视图使用户能以多种角度看待同一数据

- 视图机制能使不同用户以不同方式看待同一数据，适应数据库共享的需要



视图的作用

■ 视图对重构数据库提供了一定程度的逻辑独立性

➤ 数据库重构：

例：学生关系**Student(Sno,Sname,Ssex,Sage,Sdept)**

“垂直”地分成两个基本表：

SX(Sno,Sname,Sage)

SY(Sno,Ssex,Sdept)

通过建立一个视图**Student**：

CREATE VIEW Student(Sno,Sname,Ssex,Sage,Sdept)

AS

SELECT SX.Sno,SX.Sname,SY.Ssex,SX.Sage,SY.Sdept

FROM SX,SY

WHERE SX.Sno=SY.Sno;

使用户的外模式保持不变，用户的应用程序通过视图仍然能够查找数据



视图的作用

■ 视图对重构数据库提供了一定程度的逻辑独立性(续)

- 视图只能在一定程度上提供数据的逻辑独立性

由于对视图的更新是有条件的，因此应用程序中修改数据的

- 语句可能仍会因基本表结构的改变而改变。

■ 视图能够对机密数据提供安全保护

- 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据



视图的作用

■ 适当的利用视图可以更清晰的表达查询

- 经常需要执行这样的查询“对每个同学找出他获得最高成绩的课程号”。可以先定义一个视图，求出每个同学获得的最高成绩

```
CREATE VIEW VMGRADE
```

```
AS
```

```
SELECT Sno, MAX(Grade) Mgrade
```

```
FROM SC
```

```
GROUP BY Sno;
```

- 然后用如下的查询语句完成查询：

```
SELECT SC.Sno,Cno
```

```
FROM SC,VMGRADE
```

```
WHERE SC.Sno=VMGRADE.Sno AND
```

```
SC.Grade=VMGRADE .Mgrade;
```



视图的作用

[例15] 建立1号课程的选课视图，并要求透过该视图进行的更新操作只涉及1号课程，同时对该视图的任何操作只能在工作时间进行。

```
CREATE VIEW IS_SC
```

```
AS
```

```
SELECT Sno, Cno, Grade
```

```
FROM SC
```

```
WHERE Cno= '1'
```

```
AND TO_CHAR(SYSDATE,'HH24') BETWEEN 9 AND 17
```

```
AND TO_CHAR(SYSDATE,'D') BETWEEN 2 AND 6
```

```
WITH CHECK OPTION;
```



小结

■ SQL: 结构化查询语言

➤ DDL

- ★ Create Database

- ★ Create Table

➤ DML

- ★ Insert...Into

- ★ Update...Set...Where

- ★ Delete...from...where

- ★ Select...From...where

{
单表查询
多表查询
模糊查询
结果排序
结果去重复
条件书写



作业

■ P130:3、4、5、7、8

■ 实验



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新

下课了。。



休息。。。

